

Associate AI/ML Engineer Assignment

Objective

Build an Agentic AI + Vision system that analyzes a continuous indoor video of an elderly person and determines:

1. What the person is doing over time.
2. Whether they leave or return to bed.
3. How much time they spend in each activity/state.
4. Whether an event requires monitoring or an alert.

The emphasis is on temporal understanding, VLM/vision analysis, state tracking, and agentic decision-making, rather than building a polished application.

1. Activity / State Recognition

The system should recognize the following core states:

LYING_IN_BED
SITTING_ON_BED
SITTING_OUTSIDE_BED
STANDING
WALKING
OUT_OF_BED
UNKNOWN

Candidates may add additional states if useful but should not unnecessarily complicate the problem.

The system should understand transitions rather than independently classifying every frame.

Example:

LYING_IN_BED
↓
SITTING_ON_BED
↓
STANDING
↓
WALKING
↓
OUT_OF_BED

2. Bed Exit and Return Detection

Detect meaningful bed-related events.

Bed Exit

Lying/Sitting in bed



Standing



Moving away from bed



BED_EXIT

Return to Bed

Out of bed



Approaches bed



Sits on bed



Lies down



RETURN_TO_BED

Simply sitting up or changing sleeping position should not count as a bed exit.

3. Activity Duration

The system must calculate how long the person spends in each state.

Example output for a 20-minute video:

```
{
  "total_observation_time": "20m 00s",

  "activity_summary": {
    "lying_in_bed": "11m 42s",
    "sitting_on_bed": "2m 08s",
    "sitting_outside_bed": "1m 35s",
    "standing": "1m 03s",
    "walking": "2m 47s",
    "unknown": "45s"
  },

  "bed_summary": {
    "time_in_bed": "13m 50s",
    "time_out_of_bed": "6m 10s",
    "bed_exit_count": 2
  }
}
```

The activity durations should approximately sum to the analyzed video duration.

4. Timeline Generation

The system should produce a temporal activity timeline.

Example:

00:00 – 04:32 LYING_IN_BED
04:32 – 05:08 SITTING_ON_BED
05:08 – 05:20 STANDING
05:20 – 07:41 WALKING
07:41 – 09:15 SITTING_OUTSIDE_BED
09:15 – 09:42 WALKING
09:42 – 10:01 SITTING_ON_BED
10:01 – 15:00 LYING_IN_BED

This timeline should be generated from the detected state transitions rather than treating every frame as an independent event.

5. Agentic Analysis

The agent should decide when an observation requires more temporal context.

For example:

Observation:

Person appears beside the bed.

Agent:

Current frame is insufficient to determine whether this is a bed exit.

Action:

Analyze previous segment.

Finding:

Person was lying in bed 8 seconds earlier.

Action:

Analyze following segment.

Finding:

Person stands and walks away.

Conclusion:

BED_EXIT confirmed.

Another example:

Observation:

Person is lying horizontally.

Agent:

Determine whether the person is on the bed or floor.

Result:

Bed region + body position indicate person is lying normally in bed.

Decision:

NORMAL

6. Contextual Alert

The system should produce one of three simple decisions:

NORMAL
MONITOR
ALERT

Example:

NORMAL

Person is lying, sitting, standing, or walking normally.

MONITOR

Person sits on the edge of the bed for an unusually long period or activity cannot be confidently determined.

ALERT

A predefined safety condition occurs, such as an unexpected prolonged absence from bed.

The candidate should explain the logic behind their alert rules.

7. Expected Event Output

Example bed-exit event:

```
{
  "event": "bed_exit",
  "start_time": "00:05:08",
  "confirmed_time": "00:05:20",
  "previous_state": "sitting_on_bed",
  "current_state": "walking",
  "confidence": 0.92,
  "decision": "MONITOR"
}
```

Example complete summary:

```
{
  "observation_duration_sec": 1200,

  "activity_duration_sec": {
    "lying_in_bed": 702,
    "sitting_on_bed": 128,
    "sitting_outside_bed": 95,
    "standing": 63,
    "walking": 167,
    "unknown": 45
  },

  "bed_exit_count": 2,
  "bed_return_count": 2,

  "total_in_bed_sec": 830,
  "total_out_of_bed_sec": 370,

  "longest_out_of_bed_period_sec": 241,

  "final_state": "lying_in_bed"
}
```


8. Difficult Cases

The test data should contain some cases designed to test robustness:

- Person turning while lying in bed
- Sitting up but not leaving
- Sitting on the edge of the bed
- Standing briefly and sitting back down
- Leaving the bed
- Returning to bed
- Sitting on a chair
- Walking around the room
- Person partially hidden by blankets
- Temporary occlusion
- Caregiver entering the scene
- Poor lighting
- Person temporarily leaving camera view

The system should use UNKNOWN when there is insufficient evidence rather than forcing a classification.

9. Technical Freedom

Candidates may use any reasonable combination of:

- Vision-Language Models
- Pose estimation
- Person detection
- Person tracking
- Temporal models
- Object detection
- Bed-region detection
- Classical computer vision

Training a new model from scratch is not required.

We are interested in how the candidate combines existing models and builds a reliable system.

10. Evaluation

At minimum, evaluate:

Activity Recognition

- State classification accuracy
- Confusion between similar states
- Example failure cases

Bed Events

- Bed-exit precision
- Bed-exit recall
- False bed-exit detections

Duration Estimation

Compare predicted versus ground-truth duration for each activity.

Example:

	Ground Truth	Predicted
Lying in bed	11:50	11:42
Sitting on bed	2:00	2:08
Walking	2:52	2:47

Report duration error.

For example:

Lying duration error: 8 sec

Sitting duration error: 8 sec

Walking duration error: 5 sec

11. Deliverables

Submit:

- Source code
- README
- Simple architecture diagram
- Instructions to run the system
- Activity timeline
- Activity-duration summary
- Bed-exit/return events
- Evaluation results
- At least 3 failure-case examples

A CLI, notebook, or simple API is sufficient.

A frontend is not required.

Do not spend significant time on UI.

Candidates should be prepared to explain and modify their implementation during the interview without coding agents. Candidate should be able to justify and explain the network architectures.

Submitting

Email your repo link to careers@newnup.com with the subject line *ASE AI/ML Assignment – [Your Name]*.

Feel free to include a short note on anything you'd do differently with more time, or anything you'd want to revisit. We read those.